SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME:Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Question 1 – Online Banking: Fund Transfer Boundary Failures

Assumption: the scenario gives no figures, so I assume a transfer limit of ₹100 (minimum) to ₹1,00,000 (maximum) per transaction, a daily limit of ₹2,00,000 and an integer amount in rupees.

1. Analysis of the current strategy

- **Symptom:** failures cluster at the minimum and maximum limits. This is the classic signature of an off-by-one defect, such as using ">" where ">=" was intended.
- **Why random testing missed it:** if amounts are drawn at random from 100–1,00,000, the chance of hitting exactly 100 or 1,00,000 is about 1 in 1,00,000 per test. The boundaries are therefore effectively never tested.
- **Other gaps:** no invalid classes (zero, negative, non-numeric, empty), no specification-based expected results, and no combinations of amount with balance and daily limit.

2. Equivalence Class Partitioning (ECP)

Class	Range / condition	Type	Representative value	Expected result
EC1	Amount < 100 (and > 0)	Invalid	50	Rejected: below minimum
EC2	100 ≤ amount ≤ 1,00,000	Valid	50,000	Transfer succeeds
EC3	Amount > 1,00,000	Invalid	1,50,000	Rejected: above maximum
EC4	Amount ≤ 0	Invalid	-500 and 0	Rejected: invalid amount
EC5	Non-numeric / empty / special characters	Invalid	abc, blank, 10@0	Rejected: format error
EC6	Amount ≤ available balance	Valid	Balance 60,000, amount 50,000	Succeeds
EC7	Amount > available balance	Invalid	Balance 40,000, amount 50,000	Rejected: insufficient funds

3. Boundary Value Analysis (BVA)

Boundary	Test values	Expected result
Lower limit (100)	99, 100, 101	Reject, accept, accept
Upper limit (1,00,000)	99,999, 1,00,000, 1,00,001	Accept, accept, reject
Zero boundary	-1, 0, 1	Reject, reject, reject (below 100)
Balance boundary (balance = 5,000)	4,999, 5,000, 5,001	Accept, accept (balance becomes 0, or minimum balance rule applies), reject
Daily limit boundary (2,00,000 used cumulatively)	Total after transfer 1,99,999 / 2,00,000 / 2,00,001	Accept, accept, reject

4. Decision table for combined conditions

Condition / action	R1	R2	R3	R4
Amount within 100–1,00,000	Y	Y	Y	N
Balance sufficient	Y	N	Y	–
Daily limit not exceeded	Y	–	N	–
Action	Transfer succeeds	Reject: insufficient funds	Reject: daily limit	Reject: invalid amount

5. Is random testing worth keeping?

Random testing alone should be **replaced as the primary technique but retained as a supplement**. Its weaknesses are that it gives no guarantee of covering boundaries, it has no reliable oracle (expected result), and failures are hard to reproduce unless the random seed is stored. Its strength is that it can reveal unanticipated failures such as unusual characters, extreme lengths, or timing problems, which ECP and BVA do not target. A seeded random or fuzz run after the systematic suite therefore adds value at low cost.

6. Proposed structured approach

- **Step 1:** derive classes from the requirements (ECP), then add boundary values (BVA) for each class edge: amount, balance and daily limit.
- **Step 2:** build a decision table for the interacting conditions and write one test per rule.
- **Step 3:** add error-guessing cases such as double-click submission, session timeout during transfer, and the same amount sent twice.
- **Step 4:** automate the suite as regression tests and run it on every build; use seeded random/fuzz testing for the residual risk.

Justification: about 20 targeted tests replace thousands of random ones, give a traceable link from requirement to test, and are guaranteed to hit every boundary where the reported defects occur. This gives maximum defect detection per test executed.

Question 2 – Healthcare Module: White-Box Coverage Gaps

1. Evaluation of the current approach

Critical defects appeared after deployment because the suite verified "expected normal behaviour" (black-box, happy path) but did not measure which code was executed. No test adequacy criterion (statement, branch, path) was set, so rarely executed code such as overdose capping, allergy handling and invalid-input rejection was never run.

2. Conceptual module and control flow graph

Consider a drug-dose calculation function (illustrative, since the real code is not given):

```
1  read age, weight, allergy
2  if (age < 0 or weight <= 0)  { reject; return }        // D1
3  if (allergy == true)         { raise alert; return }   // D2
4  if (age < 12)  dose = weight * 10                      // D3
5  else           dose = weight * 8
6  if (dose > MAX) dose = MAX      // D4, with MAX = 600
7  output dose
```

CFG nodes: N1 (read), D1, N2 (reject), D2, N3 (alert), D3, N4 (paediatric dose), N5 (adult dose), D4, N6 (cap dose), N7 (output). There are 4 predicate nodes, so cyclomatic complexity V(G) = 4 + 1 = **5**, and there are **6 feasible paths**:

Path	Route	Meaning
P1	D1(T) → N2	Invalid input rejected
P2	D1(F) → D2(T) → N3	Allergy alert
P3	D1(F), D2(F), D3(T), D4(F) → N7	Child, dose within limit
P4	D1(F), D2(F), D3(T), D4(T) → N6 → N7	Child, dose capped
P5	D1(F), D2(F), D3(F), D4(F) → N7	Adult, dose within limit
P6	D1(F), D2(F), D3(F), D4(T) → N6 → N7	Adult, dose capped

3. Assessment of coverage achieved

The post-deployment defects suggest that the original suite covered only P3 and P5 (typical happy paths). For this module, that gives about 7 of 10 statements (70% statement coverage), 5 of 8 branch outcomes (about 62% branch coverage) and 2 of 6 paths (33% path coverage). These figures are illustrative, but they show the key point: even when statement coverage looks acceptable, branch and path coverage expose the untested rejection, allergy and capping logic where critical defects hide.

4. Improved strategy and test set

Test	Input (age, weight, allergy)	Path	Expected result
T1	-1, 70, no	P1	Rejected, error message
T2	30, 0, no	P1	Rejected (weight boundary)
T3	30, 70, yes	P2	Allergy alert, no dose issued
T4	5, 20, no	P3	Dose 200 (below MAX)
T5	11, 80, no	P4	Dose 800 capped at 600 (child over limit)
T6	40, 50, no	P5	Dose 400 (adult, within limit)
T7	40, 100, no	P6	Dose 800 capped at 600 (adult over limit)

- **Coverage criteria set:** 100% statement and 100% branch coverage as the minimum, plus all 6 feasible paths (and the 5 basis paths) for this safety-critical module.
- **Combine with black-box:** add boundary tests (age 11/12, dose = MAX, dose = MAX+1) so that code and specification are checked together.
- **Measure:** run a coverage tool (for example JaCoCo or gcov) in CI and make the build fail below the threshold.

5. Reliability and risk reduction

Every decision outcome is executed at least once, so hidden defects in rarely taken branches (overdose cap, allergy stop) are found before release. Test suite completeness becomes measurable and auditable, which is important for regulated healthcare software. Complete branch and path coverage reduces the probability of untested logic reaching patients, and therefore the clinical and legal risk.

Question 3 – E-commerce: Price, Quantity and Discount Combinations

Assumption: price 1–1,00,000 (₹), quantity 1–100, discount 0–50%. A discount of more than 10% is allowed only when quantity ≥ 10.

1. Limitations of the current black-box approach

- Each field was tested individually, so defects that appear only for particular **combinations** (for example high discount with low quantity) were not found.
- Only typical values were used; boundaries and invalid values were not systematically covered.
- The business rule linking quantity to the allowed discount was never expressed as a table of rules, so some rule outcomes were never tested.

2. Equivalence classes and boundaries

Field	Valid class	Invalid classes	Boundary test values
Price	1 – 1,00,000	≤ 0, > 1,00,000, non-numeric	0, 1, 2, 99,999, 1,00,000, 1,00,001
Quantity	1 – 100	≤ 0, > 100, decimal, non-numeric	0, 1, 2, 99, 100, 101
Discount (%)	0 – 50	< 0, > 50, non-numeric	-1, 0, 1, 49, 50, 51

Applying robust BVA to the three fields (one variable at a time, others nominal) gives 6n + 1 = **19 tests**. Full worst-case combination of the 7 values per field would need $7^3 = 343$ tests, which shows why a smarter technique is needed.

3. Decision table for combinations

Condition / action	R1	R2	R3	R4	R5	R6	R7	R8
Price valid	Y	Y	Y	Y	N	Y	Y	N
Quantity valid	Y	Y	Y	N	Y	Y	N	N
Discount valid (0–50)	Y	Y	N	Y	Y	Y	N	N
Discount > 10% and quantity < 10	N	Y	–	–	–	N	–	–
Action	Compute total	Reject: discount not allowed	Reject: discount invalid	Reject: quantity	Reject: price	Compute total (bulk)	Reject (multiple errors)	Reject (multiple errors)

Rule	Price, Quantity, Discount	Expected result
R1	1000, 2, 10	1800
R2	1000, 2, 25	Rejected: discount not allowed for quantity < 10
R3	1000, 2, 60	Rejected: invalid discount
R4	1000, 0, 10	Rejected: invalid quantity
R5	0, 2, 10	Rejected: invalid price
R6	1000, 10, 25	7500
R7	1000, 101, 60	Rejected: errors reported
R8	-5, -1, -1	Rejected: errors reported

4. Revised strategy and justification

- **Combine:** ECP and BVA for each field (19 tests) plus the decision table (8 rules) plus boundary of the dependency rule (discount 10/11 with quantity 9/10).
- **Pairwise testing** can be added if more fields (coupon, tax, currency) are introduced, to keep the suite small while still covering all pairs.
- **Oracle:** compute expected totals independently from the specification, and check rounding (two decimals) and the total never being negative.

Justification: the single-field approach could not detect interaction errors. The decision table makes every rule explicit, so completeness can be checked (every rule has a test), and merging “don’t care” entries removes redundant tests while ECP and BVA keep the boundaries covered.

Question 4 – Nested Conditions and Loops: Partial Path Coverage

1. Module under study (illustrative)

```
1  i = 0
2  while (i < n) {                    // P_a
3    if (x[i] > 0) {                  // P_b
4      j = 0
5      while (j < m) {                // P_c
6        if (y[j] == x[i]) {found++; break;}   // P_d
7        j++ }
8    } else neg++
9    i++ }
10 output found, neg
```

2. Evaluation of the current approach and gaps

- **CFG gaps:** testers usually draw the CFG without the loop back-edges and the "break" edge, so paths that skip or exit loops early are never identified.
- **Independent paths missed:** with 4 predicate nodes, V(G) = 4 + 1 = **5** independent paths. A team that identifies only "enter loop, run through" finds 1 or 2 of them.
- **Impact:** the untested paths (outer loop not executed, inner loop not executed, early break, negative branch) are exactly where initialisation, off-by-one and exit-condition defects occur.

3. Basis paths (complete set of independent paths)

Path	Route (by node numbers)	Test data	Expected result
BP1	1,2(F),10	n = 0	found = 0, neg = 0
BP2	1,2(T),3(F),8,9,2(F),10	n = 1, x = [-3]	neg = 1
BP3	1,2(T),3(T),4,5(F),9,2(F),10	n = 1, x = [4], m = 0	found = 0 (inner loop skipped)
BP4	1,2(T),3(T),4,5(T),6(T),break,9,2(F),10	n = 1, x = [4], y = [4]	found = 1 (early exit)
BP5	1,2(T),3(T),4,5(T),6(F),7,5(F),9,2(F),10	n = 1, x = [4], y = [9]	found = 0 (inner loop completes)

4. Loop-boundary tests

Since the number of complete paths is practically unbounded, basis-path testing is combined with Beizer's loop-testing rules. For the outer loop: n = 0, 1, 2, typical, n−1, n, and n+1 where an out-of-range case is possible. For the inner loop: m = 0, 1, 2, typical, m−1, m. Nested loops are tested from the innermost outward, holding the outer loop at its minimum value, and then the outer loop is varied.

5. Justification

- **Coverage criteria:** the 5 basis paths guarantee 100% statement and 100% branch coverage, and every edge of the CFG is exercised at least once.
- **Risk:** the loop-boundary tests cover the kind of defect (off-by-one, zero iterations) that basis paths alone cannot guarantee.
- **Expected outcome:** a finite, measurable test set (5 basis + about 8 loop tests) replaces the incomplete ad-hoc coverage, and the V(G) value gives an objective measure of when to stop.

Question 5 – Financial Application: Hybrid Test Strategy

1. Critical analysis of the current integration

- **Black-box and white-box ran independently:** black-box tests were designed from requirements and white-box tests from code, with no feedback between them. Coverage reports did not guide new black-box cases.
- **Weak adequacy criteria:** statement coverage alone was probably used. It does not show whether both outcomes of each decision, or the loop and boundary behaviour, were tested.
- **Metrics misread:** a high coverage percentage was treated as proof of quality. Coverage shows what code ran, not whether the expected result was checked or whether a missing requirement exists.
- **No traceability:** no link from requirements to test cases to code paths, so gaps stayed invisible.

2. Illustration: fee calculation

```
fee(amount):
  if (amount <= 1000)        fee = 0                    // D1
  else if (amount <= 10000)  fee = amount * 0.01        // D2
  else { fee = amount * 0.005; if (fee > 500) fee = 500 }   // D3
  return fee
```

V(G) = 3 predicates + 1 = **4** basis paths.

Technique	Test inputs	Expected fee	What it detects
ECP	500, 5000, 50000	0, 50, 250	Wrong rate per class
BVA	1000, 1001, 10000, 10001	0, 10.01, 100, 50.005 (rounded to 50.01)	Boundary off-by-one at class edges
Basis path (white-box)	500, 5000, 50000, 200000	0, 50, 250, 500	Each path incl. the cap
Boundary of the cap (from CFG)	1,00,000 / 1,00,001	500 / 500	Cap logic (fee exactly 500, and above)

The last row shows the gain from the hybrid approach: the cap threshold (amount = 1,00,000) is invisible in the requirement-based classes but is revealed by the code structure.

3. Improved hybrid strategy

Phase	Activity	Output
1	Black-box design: ECP, BVA, decision tables from requirements	Specification-based test suite
2	Execute and measure statement, branch and condition coverage	Coverage report
3	Draw CFG for uncovered code; compute V(G); derive basis-path tests	White-box tests to fill the gaps
4	Add MC/DC for critical decisions (interest, penalties, limits)	Condition-level tests
5	Mutation testing to check that tests can really detect faults	Mutation score
6	Trace requirements ↔ tests ↔ code; automate in CI	Traceability matrix; regression suite

- **Test adequacy (exit) criteria:** 100% requirement coverage, all boundaries tested, 100% branch coverage overall and 100% MC/DC/basis-path coverage for the critical calculation modules, and a mutation score above an agreed threshold.
- **Feedback loop:** uncovered code is reviewed; either a test is added or the code is shown to be dead or unreachable and removed.

4. Justification and conclusion

Black-box testing ensures the right behaviour is checked, white-box testing ensures the implemented code is exercised, and the feedback loop makes each technique compensate for the other's blind spots. Using stronger criteria than statement coverage and checking test quality with mutation testing prevents false confidence. The result is better coverage, early detection of boundary and logic defects, and defect prevention through traceability.

Question 6 – HR Payroll: Bonus Failures at 5, 10 and 15 Years

Assumed specification: bonus = salary × rate × rating multiplier. Rate by years of service: 0–4 years 2%, 5–9 years 5%, 10–14 years 8%, 15 or more years 12%. Rating (integer 1–5): 1–2 gives ×0.5, 3 gives ×1.0, 4–5 gives ×1.5. Salary used in examples: ₹50,000.

1. Failure analysis

Failures occur only at exactly 5, 10 and 15, which are the lower edges of each band. The most likely defect is a wrong relational operator, for example the code uses "years > 5" instead of "years >= 5", so these employees stay in the lower band and are underpaid. Since values like 3, 7 or 12 work correctly, the existing tests clearly sampled "typical" values inside the bands and never the edges. The test design lacked a boundary-oriented technique.

2. Equivalence Class Partitioning

Class	Years of service	Type	Representative	Expected rate
E1	< 0	Invalid	-2	Rejected
E2	0 – 4	Valid	2	2%
E3	5 – 9	Valid	7	5%
E4	10 – 14	Valid	12	8%
E5	≥ 15	Valid	20	12%
E6	Non-numeric / blank	Invalid	abc	Rejected
R1	Rating < 1 or > 5	Invalid	0, 6	Rejected
R2	Rating 1–2, 3, 4–5	Valid	2, 3, 5	×0.5, ×1.0, ×1.5

3. Boundary Value Analysis (rating = 3, salary ₹50,000)

Years	Expected rate	Expected bonus (₹)	Purpose
-1	Reject	–	Below minimum
0, 1	2%	1,000	Lower edge of first band
4	2%	1,000	Just below 5
5	5%	2,500	Boundary (reported failure)
6	5%	2,500	Just above 5
9	5%	2,500	Just below 10
10	8%	4,000	Boundary (reported failure)
11	8%	4,000	Just above 10
14	8%	4,000	Just below 15
15	12%	6,000	Boundary (reported failure)
16	12%	6,000	Just above 15

Rating boundaries, tested at years = 5: rating 0 (reject), 1 (₹1,250), 2 (₹1,250), 3 (₹2,500), 4 (₹3,750), 5 (₹3,750) and 6 (reject). Only a small number of rating × years combinations are needed because the two factors are independent in the specification, and one test at each band edge covers both.

The faulty code returns 2% at years = 5 (₹1,000 instead of ₹2,500), so the test "5 years" immediately fails and exposes the defect, and similarly for 10 and 15.

4. Why ECP + BVA is superior to exploratory testing for this logic

- **Determinism:** the defect lives at precisely known values. BVA guarantees those values are tried; exploratory testing relies on tester intuition and might miss them, as the earlier testing did.
- **Specification-based oracle:** each test has a calculated expected payout, which is essential when money is involved. Exploratory sessions typically judge "looks right".
- **Repeatability and audit:** the suite is documented, can be re-run after every change in the payroll rules, and gives evidence for audits.
- **Efficiency:** about 20 tests give systematic coverage of all bands and edges. Exploratory testing cannot demonstrate completeness.
- **Fair note:** exploratory testing remains useful for usability and unexpected scenarios, so it is a complement, not a substitute, for the financial calculation logic.

Question 7 – Laboratory Temperature Control: CFG, Branch Gaps and MC/DC

1. Module logic (illustrative)

```
1  read T, sensorFault, outOfRange       // outOfRange: T < -50 or T > 200
2  if (sensorFault) { heater=OFF; cooler=OFF }                  // D1
3  else if (T < LOW)  { heater=ON; cooler=OFF }                 // D2
4  else if (T > HIGH) { cooler=ON; heater=OFF }                 // D3
5  else { heater=OFF; cooler=OFF }
6  if (sensorFault || T > CRIT || outOfRange) alarm=ON          // D4
   else alarm=OFF
```

Specification: the alarm must be raised for any of the three conditions A = sensor fault, B = T > CRIT, C = reading outside the plausible range.

2. Control Flow Graph

Node	Description	Edges
N1	Read inputs	→ D1
D1	sensorFault?	T → N2, F → D2
N2	Switch off heater and cooler	→ D4
D2	T < LOW?	T → N3, F → D3
N3	Heater on	→ D4
D3	T > HIGH?	T → N4, F → N5
N4 / N5	Cooler on / all off	→ D4
D4	A or B or C?	T → N6 (alarm ON), F → N7 (alarm OFF)
End	Output	

There are 4 predicate nodes (D1–D4), so V(G) = 5 for the graph. However D4 is a compound decision with three conditions, so the graph hides $2^3 = 8$ combinations.

3. Gaps in branch coverage

Branch coverage requires only that each decision evaluates to both true and false once. For D4 the following two tests already give 100% branch coverage:

Test	A (fault)	B (T > CRIT)	C (range)	D4 outcome
t1	F	T	F	True
t2	F	F	F	False

Yet conditions A and C have never been tested as the reason for the alarm. A fault in how a sensor malfunction raises the alarm (for example the alarm statement placed only on the “sensor OK” path, an

operator written as && instead of ||, or short-circuit evaluation hiding the sensor condition) passes all branch tests. This is how the unmonitored sensor-malfunction sequence reached production.

4. Strategy for 100% MC/DC

MC/DC requires every condition to be shown to independently change the decision outcome, with the other conditions held fixed. For D = A || B || C, n + 1 = 4 tests are enough:

Test	A	B	C	D	Independence shown
m1	F	F	F	F	Base case (all false)
m2	T	F	F	T	A alone flips outcome (m1 vs m2)
m3	F	T	F	T	B alone flips outcome (m1 vs m3)
m4	F	F	T	T	C alone flips outcome (m1 vs m4)

- **Concrete data:** m1: sensor OK, T = 25, in range → alarm OFF. m2: sensor fault flag set, T = 25 → alarm ON. m3: T > CRIT, sensor OK → alarm ON. m4: sensor OK, reading −999 (out of range) → alarm ON.
- **Sequence tests:** sensor OK → fault → OK (alarm raised and cleared correctly), and intermittent faults, because the real failure involved a sequence.
- **Outputs:** each test must assert the heater, cooler and alarm outputs, not merely that the code ran.

5. Impact on safety-critical reliability

MC/DC is the standard expected for the highest-criticality avionics software (DO-178C, Level A) because it detects masked and incorrect conditions at a cost that grows linearly (n + 1 tests) rather than exponentially (2^n). For the lab system, it guarantees that every cause of an alarm works independently of the others, removing the class of defect that caused the failure and substantially reducing the risk to equipment, experiments and personnel.

Question 8 – Travel Portal: Promo Code Combinations

Assumed rules: a percentage promo (20%), a flat promo (₹500) and a Gold-tier discount (10%) can be combined. They are applied in the order percentage, Gold, flat; the final amount must never be below zero. The same promo cannot be applied twice.

1. Limitations of testing inputs in isolation

Each promo works correctly on its own, so isolated tests pass. The negative total only appears when the flat reduction is larger than the already-discounted price, which depends on the **interaction** of price, promo type and user tier. Isolated tests also never check the order of application or the clamp at zero.

2. Decision table

Condition / action	R1	R2	R3	R4	R5	R6	R7	R8
Percentage promo (20%)	N	Y	N	Y	N	Y	N	Y
Flat promo (₹500)	N	N	Y	Y	N	N	Y	Y
Gold tier (10%)	N	N	N	N	Y	Y	Y	Y
Price (₹)	1000	1000	1000	1000	1000	1000	1000	1000
Expected total (₹)	1000	800	500	300	900	720	400	220

Calculation example for R8: 1000 × 0.8 = 800; × 0.9 = 720; − 500 = 220.

3. Cause-effect (negative total) tests

Test	Price	Promos / tier	Raw result	Expected result
N1	300	Flat only	−200	Total = 0 (clamped, not negative)
N2	500	Flat only	0	Total = 0 (exact boundary)
N3	501	Flat only	1	Total = 1 (just above boundary)
N4	600	Percentage + Gold + Flat	−68	Total = 0
N5	1000	Same percentage code twice	–	Rejected: duplicate promo
N6	1000	Expired/invalid code + valid code	–	Invalid code rejected, valid applied

4. Optimised test set

- Full combination testing of three binary conditions gives 8 rules, and adding price classes (below flat amount, equal, above) and clamp conditions would give 8 × 3 = 24 tests.
- The clamp matters only when the flat promo is applied, so the price classes are needed only for rules R3, R4, R7 and R8 ("don't care" merging). The optimised set is **8 rule tests + 4 boundary tests (N1–N4) + 2 invalid tests (N5–N6) = 14 tests** instead of 24 or more.

- Redundant tests are removed because rules that give the same result for the same reason (percentage-only at different prices, for example) are merged into one representative.

Result: every combination of price, promo type and user tier is represented, the negative-total defect is exposed by N1 and N4, and the order-of-application and duplicate-code rules are covered with minimum duplication.

Question 9 – Legacy Migration Tool: Nested Loops and Infinite-Loop Crashes

1. Module under study (illustrative)

```
1  i = 0
2  while (i < N) {                    // record sets
3    j = 0
4    while (j < M[i]) {                // records in a set
5      if (rec[i][j].invalid) {
6        repair(rec[i][j]); continue; }  // j is NOT incremented
7      j++ }
8    i++ }
```

2. Why statement coverage was not enough

A single test with one invalid record that the repair function successfully fixes executes every statement (lines 1–8), so statement coverage reaches 100%. After the repair, the record is valid and the loop continues, so nothing seems wrong. Statement coverage does not check **how many times** a loop iterates or whether it **terminates**. If the repair fails for some record in production, the "continue" returns to line 4 with the same j and the loop never ends. A statement-level metric cannot detect this, because it only asks whether each line ran at least once.

3. Cyclomatic complexity

Predicate nodes: line 2, line 4 and line 5 = 3. V(G) = P + 1 = **4**, so four independent paths must be tested (equivalently, E − N + 2 from the graph).

Path	Description	Test data	Expected result
BP1	Outer loop skipped	N = 0	Terminates immediately
BP2	Outer once, inner skipped	N = 1, M[0] = 0	Terminates, no records processed
BP3	Inner iterates, record valid	N = 1, M[0] = 1, valid record	Terminates, j increments
BP4	Invalid record, repair succeeds, continue	N = 1, M[0] = 1, invalid, repair OK	Record repaired and loop exits

4. Loop-boundary and termination tests

Test	Scenario	Expected
L1	Inner loop 0, 1, 2, typical, max−1, max iterations	Correct count and termination
L2	Outer loop 0, 1, 2, typical, max iterations	Correct count and termination
L3	Repair always fails for one record (**the production failure**)	Test must time out or the code must stop with an error: the defect is exposed
L4	Many consecutive invalid records, all repaired	Terminates, linear time
L5	Empty set between non-empty sets	Outer loop moves on correctly

5. Justification

- **Cyclomatic complexity** gives the minimum number of paths (4) that must be exercised and a measurable stopping criterion, and a value that grows with nesting warns of testing difficulty and refactoring need.
- **Test L3** is the crucial gap: it forces the "continue without increment" path and uses a timeout so the infinite loop shows up as a test failure instead of a production crash.
- **Fix and defence:** increment j (or count repair attempts and abort after a limit) so that every path guarantees progress; keep the loop-bound tests as regression checks.

Question 10 – Food Delivery App: State Transition Testing

1. Evaluation of the current black-box strategy

ECP and BVA treat inputs as independent values and ignore **history**: the same event ("driver cancels") should have a different outcome depending on the current state. The team tested valid inputs in the normal order Ordered → Preparing → Out for Delivery → Delivered. No behaviour is defined for "driver cancels" while Out for Delivery, so the app has no outgoing transition and remains stuck.

2. State model (with the missing state added)

States: **Ordered, Preparing, Out for Delivery, Reassigning Driver** (new), **Delivered** (final), **Cancelled** (final). Events: restaurant accepts (RA), food picked up (PU), delivered (DL), customer cancels (CC), driver cancels (DC), restaurant cancels (RC), no driver found (ND), new driver accepts (NDA).

Current state	**RA**	**PU**	**DL**	**CC**	**DC**	**RC**	**ND**	**NDA**
Ordered	Preparing	Invalid	Invalid	Cancelled	Ignore	Cancelled	–	–
Preparing	Invalid	Out for Delivery	Invalid	Cancelled (fee)	Preparing (reassign)	Cancelled	–	–
Out for Delivery	Invalid	Invalid	Delivered	Cancelled (policy)	**Reassigning Driver**	Invalid	–	–
Reassigning Driver	Invalid	Invalid	Invalid	Cancelled (refund)	Ignore	Invalid	Cancelled (refund)	Out for Delivery
Delivered, Cancelled	Invalid	Invalid	Invalid	Invalid	Invalid	Invalid	Invalid	Invalid

3. Redesigned test scenarios

Scenario	Event sequence	Expected final state / behaviour
S1 (happy path)	RA, PU, DL	Delivered
S2	Customer cancels while Ordered	Cancelled, full refund
S3	RA, driver cancels in Preparing	Stays Preparing, new driver assigned
S4 (reported bug)	RA, PU, DC, NDA, DL	Out for Delivery → Reassigning → Out for Delivery → Delivered
S5	RA, PU, DC, ND (timeout)	Cancelled, refund, customer notified
S6	RA, PU, DC, NDA, DC (second cancellation)	Reassigning again, no deadlock
S7 (invalid events)	DL while Ordered; RA while Delivered	Event rejected, state unchanged
S8	App closed or network lost while Reassigning, then reopened	State restored correctly from the server

Coverage criteria: all states (6/6), all valid transitions (0-switch coverage), all invalid state/event pairs from the table (negative testing), and selected 1-switch sequences (two consecutive transitions) such as DC → NDA → DC.

4. Why this technique fills the gaps left by ECP and BVA

- **Sequence dependence:** the failure needs a specific order of events. ECP/BVA partition single inputs, so they cannot represent "the same event in a different state".
- **Unspecified behaviour:** building the state/event table forces every cell to be considered, so the missing "DC while Out for Delivery" transition shows up as an empty cell during design.
- **Dynamic workflows:** concurrency, timeouts and restoring state after app restarts are properties of the state machine, not of any input value.
- **Measurable completeness:** the table provides a clear, countable coverage goal, unlike exploratory use of the app.

Question 11 – Authentication Module: Hybrid Testing for Brute-Force Weakness

1. Assessment of the black-box and white-box methods used

- **Black-box:** tests probably verified correct and incorrect credentials, a valid and an expired OTP, and a trusted and an unknown IP. These are functional checks of single factors and do not test repeated or abusive use.
- **White-box:** coverage was measured on the success path and a few failure paths. No one checked that **every failure path** updates the failed-attempt counter, which is a property of the code structure, not of an input value.
- **Gap:** brute-force resistance is a non-functional, sequence-dependent property. Neither technique alone targets it.

2. Module model and path analysis (illustrative)

```
D1  if (locked)              -> reject
D2  if (!passwordCorrect)    -> failCount++; goto D6
D3  if (!otpValid)           -> (counter not changed?)  // suspect path
D4  if (otpExpired)          -> reject
D5  if (!ipAllowed)          -> step-up / reject; else grant access
D6  if (failCount >= LIMIT)  -> locked = true
```

There are 6 predicate nodes, so V(G) = 6 + 1 = **7** independent paths. For **each** path, the tester asks a security question: is a failed attempt counted, does the counter survive a change of IP or a new session, is lock-out enforced, and is anything revealed to the user about which factor failed?

Path	Route	Security property to verify
SP1	D1 true	Locked account stays locked, even with the correct password and OTP
SP2	D2 false → D6 below limit	Failed password is counted
SP3	D2 false → D6 at limit	Account is locked at exactly LIMIT attempts (boundary: LIMIT−1, LIMIT, LIMIT+1)
SP4	D3 false (wrong OTP)	Wrong OTP is also counted; OTP attempts are rate-limited (**path analysis exposes a missing counter update here**)
SP5	D4 true (expired OTP)	Expired OTP rejected; old OTP cannot be reused
SP6	D5 false (new IP)	Extra verification required; counter not reset by IP change
SP7	All true	Successful login resets counter; session created securely

3. Error-guessing tests (black-box, experience-based)

Idea	Test design (defensive)	Expected behaviour
Rapid repeated attempts	Many wrong passwords in a short time	Lock-out or progressive delay after LIMIT
Counter reset	Alternate wrong attempts with IP/device change or page reload	Counter is per account, not per session or IP
OTP timing	OTP at 29, 30 and 31 seconds (30 s validity assumed)	Valid, valid, rejected
OTP reuse / guessing	Reuse a used OTP; many wrong OTPs	Rejected; OTP attempts limited
Concurrent requests	Parallel login requests for the same account	Counter is updated atomically (no race condition)
Input abuse	Leading/trailing spaces, very long strings, special characters	Handled safely, no error leakage
Information leakage	Compare messages for wrong user vs wrong password	Same generic message

4. Hybrid strategy and justification

- **Step 1:** build the CFG and basis paths (7 paths); for each path attach security assertions as expected results.
- **Step 2:** use error guessing to create abuse sequences and boundary values (attempt count LIMIT−1/LIMIT/LIMIT+1, OTP 29/30/31 s), then run them against the paths they exercise.
- **Step 3:** use coverage to find paths that no abuse test reaches, and add error-guessing cases for them. Add security tools (rate-limit and fuzz tests) as a complement.

Justification: path analysis guarantees that every branch of the authentication logic is examined (including failure paths that update no counter), while error guessing brings in the attacker's viewpoint, with realistic sequences and timing that formal techniques do not generate. Together they close the gap that allowed brute-force weaknesses, and improve the security posture through systematic, repeatable, risk-driven testing.

Question 12 – E-learning Platform: Tiered Access and Expiry-Day Upgrades

Assumption: a subscription is valid from the start date up to and including the expiry date (23:59:59 server time). Free users access Free courses, Basic users access Free and Basic courses, and Premium users access all courses.

1. Adequacy of current criteria

The failure on the exact day of expiration means the existing criteria did not require tests at date boundaries, nor tests of **state changes** (upgrades) combined with dates. Testing seems to have used mid-period dates and single tiers. The criterion "each tier and course type has been tried once" is too weak: it does not require coverage of every date class or every upgrade path.

2. Equivalence classes

Dimension	Classes	Representative values
Subscription tier	Free, Basic, Premium	One of each
Course type	Free, Basic, Premium	One of each
Date relative to expiry (D)	D1: before start (invalid); D2: active period; D3: day before expiry; D4: **expiry day**; D5: after expiry (grace/lapsed)	start − 1 day; mid-period; expiry − 1; expiry (00:00:00 and 23:59:59); expiry + 1
Upgrade action	Free→Basic, Free→Premium, Basic→Premium, same-tier renewal, downgrade	Each performed in classes D2, D3, D4, D5

3. Test design

Test	Tier / date	Action	Expected result
A1	Basic, mid-period	Open Basic course	Access granted
A2	Basic, mid-period	Open Premium course	Access denied, upgrade prompt
A3	Basic, expiry − 1	Open Basic course	Access granted
A4	Basic, expiry day 23:59:59	Open Basic course	Access granted (still valid)
A5	Basic, expiry + 1 (00:00:00)	Open Basic course	Access denied or revert to Free (per rule)
U1	Basic, expiry − 1	Upgrade to Premium	Premium access immediately; new expiry computed from the upgrade date
U2	Basic, **expiry day**	Upgrade to Premium	Premium access without loss of progress; no gap or double-billing
U3	Basic, expiry day 23:59:59, upgrade completes after midnight	Upgrade	Defined behaviour: treated as upgrade, not as lapsed re-subscription
U4	Basic, expiry + 1	Upgrade to Premium	Treated as new subscription
U5	Free, any day	Upgrade to Basic / Premium	Correct access and start/expiry dates
U6	Premium, expiry day	Downgrade to Basic	Defined by rule (effective at renewal), no premium content lost early
Z1	User and server in different time zones (IST vs UTC)	Expiry day tests A4 and U2	Same result in both

4. Justification of completeness

- **All classes covered:** every tier, course type, date class and upgrade path appears in at least one test (tier × date class, 3 × 5 = 15 combinations are covered by A1–A5, U1–U6 plus pairwise additions).
- **Boundaries included:** the expiry date is tested on both sides (expiry − 1, expiry, expiry + 1) and at the edges of the day, which is where the reported defect occurs.
- **Transitions tested, not just states:** upgrades are tested in each date class, so user transitions are seamless (access, billing, progress and dates stay consistent).

Question 13 – Smart-home Lighting Controller: Branch and Condition Coverage

1. Module (illustrative)

```
switch (mode) {
  case AUTO:
    if (ambient < 30 && (motion || scheduleOn) && !override)
        level = dimFor(ambient);
    else level = 0;  break;
  case MANUAL: level = userLevel;  break;
  case NIGHT:  level = 10;         break;
  default:     level = SAFE;
}
```

2. Risks of relying on statement coverage

- One AUTO test where the condition is true, plus one test each for MANUAL and NIGHT executes every statement in the cases, but the **else branch**, the **default** case and **false outcomes** of the compound condition would not necessarily be run.
- Short-circuit evaluation: with "ambient < 30 && …" the later conditions are not evaluated if ambient ≥ 30, so errors in them stay hidden.
- Sensor noise and unexpected values (negative or saturated lux readings, invalid mode) occur in the default and false paths, which cause hardware-software integration problems such as flicker, lights stuck at full, or lights off at night.

3. Branch coverage

Decision	Outcome needed	Test
switch(mode)	AUTO, MANUAL, NIGHT, default (4 outcomes)	One test each, including invalid mode (e.g., 99)
AUTO if-condition	True and False	Ambient 20, motion, no override → dims; ambient 80 → level 0

4. Condition coverage and MC/DC for the AUTO decision

Condition coverage requires every atomic condition (A = ambient < 30, B = motion, C = scheduleOn, O = override) to be true and false at least once. Branch + condition coverage may still not show that each condition independently matters. With 4 conditions, testing all combinations (multiple-condition coverage) would need $2^4 = 16$ tests; MC/DC needs only n + 1 = **5**:

Test	A	B	C	O	Decision A && (B||C) && !O	Independent effect shown
c1	T	T	F	F	True	Base
c2	F	T	F	F	False	A (vs c1)
c3	T	F	F	F	False	B (vs c1)
c4	T	F	T	F	True	C (vs c3)
c5	T	T	F	T	False	O (vs c1)

Missing logical paths revealed by this analysis: schedule-only activation (c4), override blocking an otherwise true decision (c5), and the false outcomes caused by a single failing condition (c2, c3).

5. Improved white-box strategy and risk reduction

- **Test set:** 4 switch outcomes + 5 MC/DC tests + boundary values on the sensor (ambient = 29, 30, 31; 0; negative; maximum reading) + sequence tests for flicker (rapidly changing values around 30 lux, to check hysteresis if specified).
- **Criteria:** 100% statement, 100% branch, 100% condition and MC/DC for the AUTO decision, measured with a coverage tool.
- **Risk:** each condition and each switch outcome including default is demonstrated to work, so unexpected sensor data or mode values lead to a defined, safe output instead of undefined hardware behaviour at integration time.

Question 14 – Logistics System: Zero and Negative Weight Inputs

Assumption: valid weight is 0.1 kg to 50 kg, zone 1–4, speed Standard or Express. Cost = (base rate per zone + weight × per-kg rate) × speed factor.

1. Why the input validation testing failed

- Test data contained only realistic, valid shipments. Nobody tried values that "should never happen", so missing validation was never noticed.
- The normal BVA used only the valid range (min, max) and did not extend to values just outside it.
- With weight 0 the formula still returns the base rate (a free or underpriced shipment), and with a negative weight the cost may drop below the base rate or even become negative, which causes direct financial loss, and wrong data reaches billing and routing.

2. Robustness boundary value analysis for weight

Value	Boundary position	Expected result
-1, -0.1	Negative, invalid	Rejected: "weight must be positive"
0	Zero, invalid	Rejected: "weight must be at least 0.1 kg"
0.09	Min − δ	Rejected
0.1	Minimum (valid)	Cost calculated
0.11	Min + δ	Cost calculated
25	Nominal	Cost calculated
49.9	Max − δ	Cost calculated
50	Maximum (valid)	Cost calculated
50.01	Max + δ	Rejected: exceeds limit (or routed to freight)
abc, blank, null, NaN, 1e308, 2.555555	Format / type / precision	Rejected or rounded as specified; no crash

For several inputs, robust worst-case combinations are added for the other parameters: invalid zone (0, 5, blank) and invalid speed (null, unknown value) with a valid weight, and one invalid input at a time so that each rejection can be linked to its cause. A few multi-fault cases (negative weight and invalid zone) check that errors are reported and nothing is calculated.

3. Cost correctness tests (valid values)

Weight	Zone	Speed	Expected
0.1	1	Standard	Base_1 + 0.1 × rate, minimum charge applied if specified
50	4	Express	Maximum cost, no overflow
10	2	Express	Standard × express factor

4. Why negative scenarios are necessary

- **Financial integrity:** invalid values must never produce a price, particularly a zero or negative price.
- **Robustness:** real users and other systems send bad data; a robust system rejects it with a clear message instead of crashing or silently accepting it.
- **Security:** missing validation is a route for injection and tampering with API parameters.
- **Defect economics:** most input-handling defects are found only by negative tests, and the cost of fixing them after release (wrong invoices, chargebacks) is much higher than writing about ten extra tests.

Question 15 – Medical Imaging: Parallel Loops and Prime Data Sizes

1. Evaluation of random testing

Random testing treats the module as a black box and draws sizes without regard to its structure. Here the failure region is defined by arithmetic: a work-splitting step divides the data by the number of threads, and a prime size n > T (T = threads) is **never** divisible by T, so a remainder chunk exists. If that remainder is dropped, duplicated or mis-indexed, the result is wrong only for sizes with a non-zero remainder or particular small remainders. Random sizes rarely test the special values (n = 0, 1, T−1, T, T+1, or sizes where n mod T is 1 or T−1), they provide no oracle for "correct" image results, and parallel faults are non-deterministic so a random pass says little. Random testing is therefore not adequate as the main technique for this mathematical module.

2. Code structure and Basis Path Testing

```
1  if (n == 0) return empty                    // D1
2  chunk = n / T;  rem = n % T
3  for (t = 0; t < T; t++) {                   // D2 (loop)
4     start = t * chunk
5     end   = start + chunk
6     if (t == T - 1 && rem != 0) end = n      // D3 (compound)
7     spawn process(start, end) }
8  join all; combine results
```

Predicates: D1, D2, D3 (with 2 conditions, counted as 2 predicates) → P = 4. V(G) = 4 + 1 = **5** basis paths:

Path	Description	Test data (T = 4)	Expected result
BP1	n = 0, early return	n = 0	Empty result, no thread started
BP2	Loop runs, rem = 0 (divisible)	n = 16	All 16 elements processed once
BP3	Last thread, rem ≠ 0 (**prime n**)	n = 17, 13, 7919	All elements processed, last chunk extended to n
BP4	n < T (chunk = 0)	n = 3	All 3 elements processed, no empty-range crash
BP5	Last-thread condition false, rem ≠ 0 (t ≠ T−1)	n = 18 (rem 2), checked for non-last threads	Non-last chunks have equal size and do not overlap

3. Additional structured tests

- **Boundary sizes:** n = 1, T−1, T, T+1, k·T−1, k·T, k·T+1, and primes (2, 3, 5, 7, 13, 101, 7919). Also vary the thread count T = 1, 2, 3, 8, and T larger than n.
- **Oracle and invariants:** compare the parallel result with a sequential reference (differential testing); check that the chunk lengths sum to n, ranges do not overlap and no index is skipped. Floating-point sums are compared with a tolerance because the order of addition changes with T.

- **Concurrency:** run each case repeatedly (for example 100 times) with different scheduling, and use a thread sanitizer to detect data races, which show up as intermittent wrong results.
- **Property-based/random testing as a supplement:** generate sizes, but include prime and boundary values by construction.

4. Justification

- **Mathematical accuracy:** the invariants and the sequential-reference comparison check the numerical result, not only that the code ran. Prime sizes ($n \bmod T \neq 0$) are directly targeted by BP3.
- **Stability:** basis paths (5 paths, 100% statement and branch coverage) plus boundary sizes and repeated parallel runs cover the structural and concurrency causes of inconsistent output.
- **Safety:** in medical imaging, an incorrect result may lead to a wrong diagnosis; a measurable, systematic approach reduces this risk far better than random sampling.

Code of Conduct:

*I **P.Paneendra (192321072)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:P.Paneendra

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification